Documents de spécifications (encoding vectors)

RENDU Etape intermédiaire (Adam,Ayman,Tom)

Instalation:

Action	Commande
clonner le projet	`git clone git@github.com:yahia-adam/deepRL_5iabd.git && cd deepRL_5iabd`
Installer les dépendances	`uv sync`
Activer l'environement	`source .venv/bin/activate`
Lancer un script	`uv run python -m mypythonlib.envs.quarto`

Line world

Jeu avec Joueur humain + GUI

```
PYTHONPAH=. uv run python3 -m deeprl_5iabd.envs.line_world
```

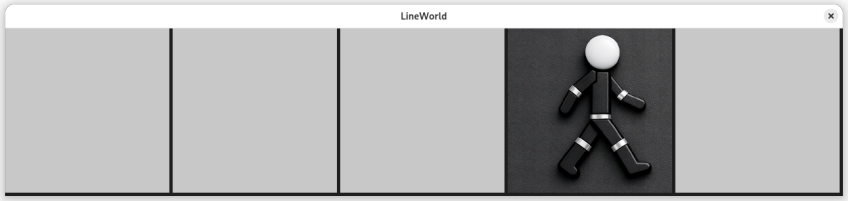

Figure 1: alt text

Proposition de description de l'état du jeu (vecteur d'encoding)

vecteur de 1 valeur la position de l'agent, pourquoi un vecteur pour que tous mes envs aille les meme methode avec les meme signature (baseEnv clasee parent)

```
def get_observation_space(self) -> list[int]:
    return [self.agent_pos]
```

Proposition de description d'une action du jeu (vecteur d'encoding)

vecteur de 2 valeurs [gauche, droit] - [0,1] quand l'agent est a la case 0 - [1,1] quand l'agent est entre 1 et la case 3 - [0,1] quand l'agent est a la case 4

```
    def get_action_space(self) -> list[int]:
        """0: gauche, 1: droite"""
        action = [1,1]
        if self.agent_pos == 0:
            action[0] = 0
        if self.agent_pos == 4:
            action[1] = 0
        return action
```

Grid World

Jeu avec Joueur humain + GUI

```
PYTHONPATH=. uv run python3 -m deeprl_5iabd.envs.grid_world
```

Proposition de description de l'état du jeu (vecteur d'encoding)

Vecteur de **25 valeurs** représentant le plateau 5×5, rempli de `-1` partout sauf à la position de l'agent où la valeur est `1`.

```
def get_observation_space(self):
    return self.board.flatten().tolist()
```

Proposition de description d'une action du jeu (vecteur d'encoding)

Vecteur de **4 valeurs** `[bas, haut, droite, gauche]`, chaque valeur vaut `1` si l'action est disponible, `0` sinon.

Exemples : - `[1, 0, 1, 0]` quand l'agent est en haut à gauche (case (0,0)) - `[1, 1, 1, 1]` quand l'agent est au centre (case (2,2))

```
def get_action_space(self) -> list[int]:
    picks = np.ones(4)
    if self.agent_pos[0] == 0:
        picks[1] = 0
    if self.agent_pos[1] == 0:
        picks[3] = 0
    if self.agent_pos[0] == self.BOARD_SIZE - 1:
        picks[0] = 0
    if self.agent_pos[1] == self.BOARD_SIZE - 1:
```

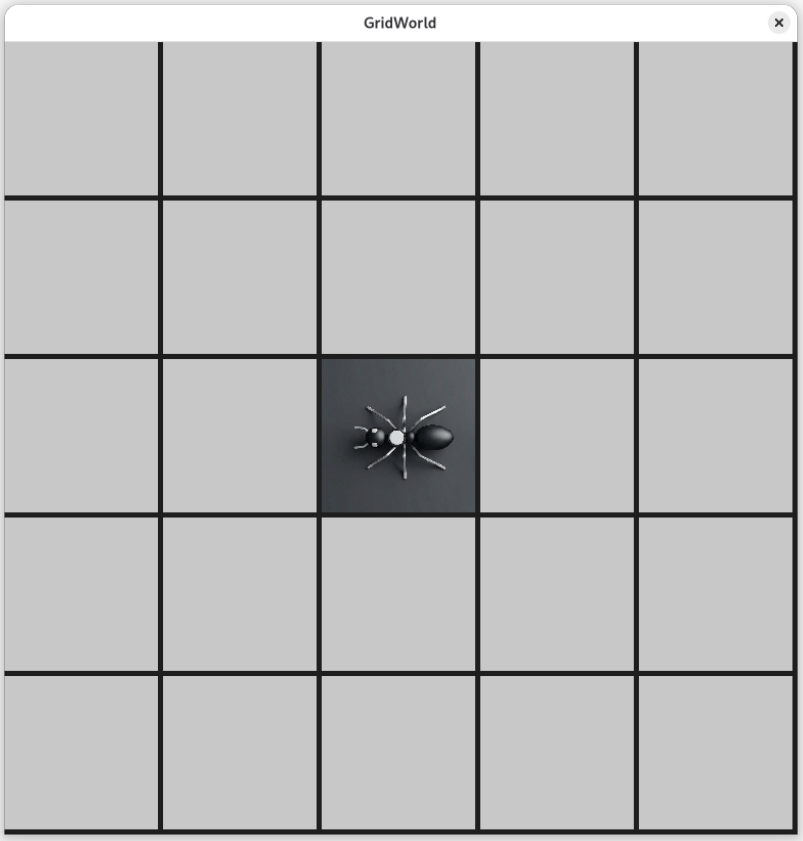

Figure 2: alt text

```
        picks[2] = 0
    return picks.tolist()
```

Tic-Tac-Toe

Jeu avec Joueur humain + GUI

```
PYTHONPATH=. uv run python3 -m deeprl_5iabd.envs.tictactoe
```

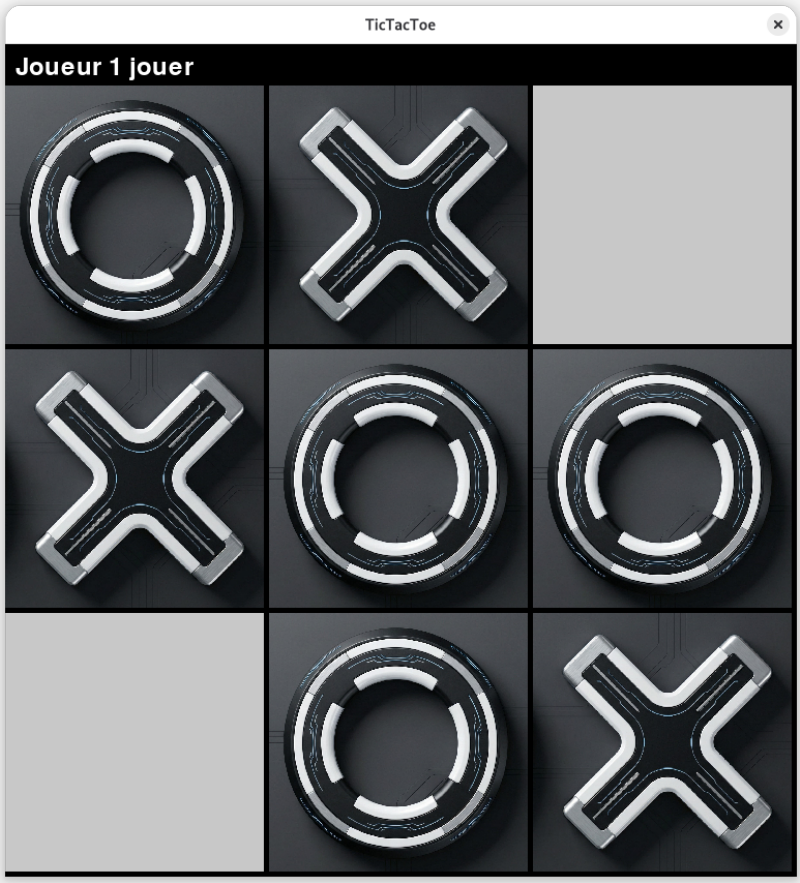

Figure 3: alt text

Proposition de description de l'état du jeu (vecteur d'encoding)

Vecteur de **9 valeurs** représentant le plateau 3×3: - `-1` quand la case est vide - `0` quand la case est occupée par le joueur O (joueur 0) - `1` quand la case est occupée par le joueur X (joueur 1)

```
def get_observation_space(self) -> np.ndarray:
    return self.board.flatten().tolist()
```

Proposition de description d'une action du jeu (vecteur d'encoding)

Vecteur de **9 valeurs**, une par case du plateau : - `1` quand la case est **vide** (action jouable) - `0` quand la case est **occupée** (action non disponible)

```
    def get_action_space(self):
        return (self.board.flatten() == -1).astype(int)
```

Simulation de jeu avec joueur random (calculer le nombre de parties / seconde)

```
PYTHONPAH=. uv run python3 -m deeprl_5iabd.training.envs_per_counter
```

On teste 1 000 000 parties jouées par un agent aléatoires sur chaque environnement.

- **LineWorld / GridWorld** : ~15–20M parties/sec.
- **TicTacToe** : ~26K parties/sec.
- **Quarto** : ~38K parties/sec, légèrement plus rapide que TicTacToe je sais pas Pourquoi :).

```
┌─(~/Documents/esgi/drl/deepRL_5iabd)──────────────────────────────────────────────────────────────────────────────
└─(00:24:14 on main • ⋆)──> PYTHONPAH=. uv run python3 -m deeprl_5iabd.training.envs_per_counter
pygame 2.6.1 (SDL 2.28.4, Python 3.12.12)
Hello from the pygame community. https://www.pygame.org/contribute.html


TEST SUR 1000000 PARTIES ...
-------------------------------------------
Environnement         Durée (s)   Matchs/sec
-------------------------------------------
line_world                 0.05     19697311
GridWorld                  0.07     15143620
TicTacToe                 38.35        26074
quarto                    25.98        38493
┌─(~/Documents/esgi/drl/deepRL_5iabd)──────────────────────────────────────────────────────────────────────────────
└─(00:27:22 on main • ⋆)──> 
```

Figure 4: alt text

```
def count_n_match_time(env: BaseEnv, num_episode):
    player = RandomPlayer(action_dim=len(env.get_action_space()))
    s = perf_counter()
```

```
i = 0
while (i <= num_episode):
    while (not env.is_game_over()):
        action_spaces = env.get_action_space()
        probs = player.forward(x=None, mask=action_spaces)
        probs_dist = Categorical(probs)
        action_pos = probs_dist.sample()
        env.step(action_pos.item())
    i += 1
e = perf_counter()

total_time = e - s
match_per_s = num_episode / total_time
return total_time, match_per_s
```